Pr. BE ELBAGHAZAOUI

ENSA BM

Rapport de TP

TP 1 : Application Monolithique avec Spring Boot

Gestion des Produits

Nom et prénom : BOUHAMIDI-ALAOUI Khaoula
Filière : IACS1
Encadrant : Pr. BE ELBAGHAZAOUI
Établissement : ENSA BM

Table des matières

1 Introduction **2**

2 Technologies utilisées **2**

3 Étapes de réalisation **2**
3.1 Création du projet 2
3.2 Configuration du fichier application.properties 3
3.3 Création de l'entité Product 3
3.4 Création du Repository 4
3.5 Création du Service 4
3.6 Création du Controller 5
3.7 Lancement de l'application 5

4 Test avec Postman **6**
4.1 Test de la méthode GET 6
4.2 Test de la méthode POST 6
4.3 Test de la méthode PUT 7
4.4 Test de la méthode DELETE 8

5 Exercice : gestion des catégories **8**
5.1 Création de l'entité Category 8
5.2 Modification de l'entité Product 9
5.3 Création de CategoryRepository 9
5.4 Création de CategoryController 10
5.5 Test avec Postman 11

6 Conclusion **11**

1 Introduction

Ce TP a pour objectif de développer une API REST en utilisant Spring Boot afin de gérer des produits via les opérations CRUD (Create, Read, Update, Delete). L'application est connectée à une base de données PostgreSQL et testée à l'aide de Postman.

2 Technologies utilisées

- Spring Boot
- PostgreSQL
- Maven
- IntelliJ IDEA
- Postman

3 Étapes de réalisation

3.1 Création du projet

Nous avons créé un projet Spring Boot avec une architecture organisée en couches : model, repository, service et controller.

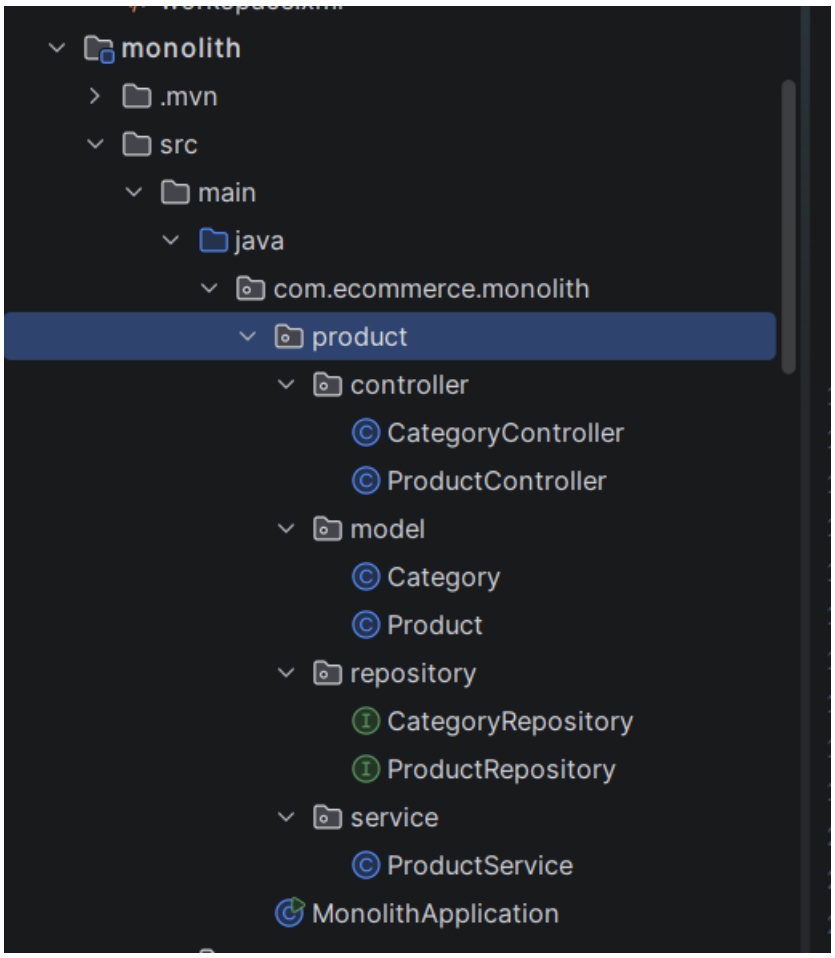

FIGURE 1 – Structure du projet

3.2 Configuration du fichier application.properties

Dans cette étape, nous avons configuré le fichier application.properties afin d'assurer la connexion entre l'application Spring Boot et la base de données PostgreSQL. Nous avons défini l'URL de connexion, le nom d'utilisateur, le mot de passe ainsi que les propriétés Hibernate nécessaires à la création et la mise à jour automatique des tables.

```
application.properties × | Product.java | ProductRepository.java | Produ

spring.application.name=monolith

spring.datasource.url=jdbc:postgresql://localhost:5432/ecommerce
spring.datasource.username=postgres
spring.datasource.password=postgres

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

FIGURE 2 – Configuration du fichier application.properties

3.3 Création de l'entité Product

Nous avons créé une entité Product contenant les attributs name, description, price et stock. Le type BigDecimal a été utilisé pour assurer une meilleure précision du prix, et des annotations de validation ont été ajoutées pour contrôler les données saisies.

```
Product.java × | ProductService.java | ProductRepository.java | Produ

@Table(name = "products")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "Product name is required")
    @Column(nullable = false)
    private String name;

    @Column(length = 1000)
    private String description;

    @NotNull(message = "Price is required")
    @Positive(message = "Price must be greater than 0")
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal price;

    @NotNull(message = "Stock is required")
    @PositiveOrZero(message = "Stock cannot be negative")
    @Column(nullable = false)
    private Integer stock;
}
```

FIGURE 3 – Classe Product.java

3.4 Création du Repository

Le repository permet d'accéder aux données via JpaRepository. Il simplifie les opérations de persistance en évitant l'écriture manuelle des requêtes SQL de base.

```java
package com.ecommerce.monolith.product.repository;

import com.ecommerce.monolith.product.model.Product;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;

public interface ProductRepository extends JpaRepository<Product, Long> {

    List<Product> findByNameContaining(String name);
}
```

FIGURE 4 – ProductRepository

3.5 Création du Service

La couche Service permet de gérer la logique métier et de séparer le contrôleur de l'accès direct au repository. Elle contient les méthodes nécessaires pour récupérer, ajouter, modifier et supprimer les produits.

```java
package com.ecommerce.monolith.product.service;

import com.ecommerce.monolith.product.model.Product;
import com.ecommerce.monolith.product.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
@RequiredArgsConstructor
@Transactional
public class ProductService {

    private final ProductRepository repository;

    @Transactional(readOnly = true)
    public List<Product> getAll() {
        return repository.findAll();
    }

    @Transactional(readOnly = true)
    public Product getById(Long id) {
        return repository.findById(id)
                .orElseThrow(() -> new IllegalArgumentException("Product not found with id: " + id));
    }

    public Product create(Product product) {
        return repository.save(product);
    }
```

FIGURE 5 – ProductService

3.6 Création du Controller

Le contrôleur expose les endpoints REST nécessaires pour effectuer les opérations CRUD sur les produits. Il reçoit les requêtes HTTP et appelle les méthodes du service.

```java
13  @RequiredArgsConstructor
14  @Validated
15  public class ProductController {
16      private final ProductService service;
17      @GetMapping
18      public ResponseEntity<List<Product>> getAll() {
19          List<Product> products = service.getAll();
20          return ResponseEntity.ok(products);
21      }
22      @GetMapping("/{id}")
23      public ResponseEntity<Product> getById(@PathVariable Long id) {
24          Product product = service.getById(id);
25          return ResponseEntity.ok(product);
26      }
27      @PostMapping
28      public ResponseEntity<Product> create(@Valid @RequestBody Product product) {
29          Product created = service.create(product);
30          return ResponseEntity.status(HttpStatus.CREATED).body(created);
31      }
32      @PutMapping("/{id}")
33      public ResponseEntity<Product> update(@PathVariable Long id,
34                                            @Valid @RequestBody Product product) {
35          Product updated = service.update(id, product);
36          return ResponseEntity.ok(updated);
37      }
38      @DeleteMapping("/{id}")
39      public ResponseEntity<Void> delete(@PathVariable Long id) {
40          service.delete(id);
41          return ResponseEntity.noContent().build();
42      }
43  }
```

FIGURE 6 – ProductController

3.7 Lancement de l'application

Après l'implémentation des différentes classes, l'application a été lancée avec succès. Le serveur embarqué Tomcat a démarré sur le port 8080, ce qui confirme le bon fonctionnement du projet.

```
Run   MonolithApplication

 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/

 :: Spring Boot ::                (v4.0.3)

2026-03-21T19:28:43.065Z  INFO 27252 --- [monolith] [           main] c.e.monolith.MonolithApplication         : Starting MonolithApplication using Java 17.0.18 with PID 27252
2026-03-21T19:28:43.069Z  INFO 27252 --- [monolith] [           main] c.e.monolith.MonolithApplication         : No active profile set, falling back to 1 default profile: "def
2026-03-21T19:28:43.781Z  INFO 27252 --- [monolith] [           main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data JPA repositories in DEFAULT mode.
2026-03-21T19:28:43.843Z  INFO 27252 --- [monolith] [           main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data repository scanning in 52 ms. Found 1 JPA
2026-03-21T19:28:44.347Z  INFO 27252 --- [monolith] [           main] o.s.boot.tomcat.TomcatWebServer          : Tomcat initialized with port 8080 (http)
2026-03-21T19:28:44.362Z  INFO 27252 --- [monolith] [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-03-21T19:28:44.363Z  INFO 27252 --- [monolith] [           main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/11.0.18]
2026-03-21T19:28:44.424Z  INFO 27252 --- [monolith] [           main] b.w.c.s.WebApplicationContextInitializer : Root WebApplicationContext: initialization completed in 1295 m
2026-03-21T19:28:44.592Z  INFO 27252 --- [monolith] [           main] org.hibernate.orm.jpa                    : HHH008540: Processing PersistenceUnitInfo [name: default]
2026-03-21T19:28:44.645Z  INFO 27252 --- [monolith] [           main] org.hibernate.orm.core                   : HHH000001: Hibernate ORM core version 7.2.4.Final
2026-03-21T19:28:45.141Z  INFO 27252 --- [monolith] [           main] o.s.o.j.p.SpringPersistenceUnitInfo      : No LoadTimeWeaver setup: ignoring JPA class transformer
2026-03-21T19:28:45.176Z  INFO 27252 --- [monolith] [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Starting...
2026-03-21T19:28:45.373Z  INFO 27252 --- [monolith] [           main] com.zaxxer.hikari.pool.HikariPool        : HikariPool-1 - Added connection org.postgresql.jdbc.PgConnecti
2026-03-21T19:28:45.376Z  INFO 27252 --- [monolith] [           main] com.zaxxer.hikari.HikariDataSource       : HikariPool-1 - Start completed.
2026-03-21T19:28:45.455Z  INFO 27252 --- [monolith] [           main] org.hibernate.orm.connections.pooling    : HHH10001005: Database info:
    Database JDBC URL [jdbc:postgresql://localhost:5432/ecommerce]
    Database driver: PostgreSQL JDBC Driver
    Database dialect: PostgreSQLDialect
    Database version: 18.0
    Default catalog/schema: ecommerce/public
    Autocommit mode: undefined/unknown
```

FIGURE 7 – Console de démarrage

4 Test avec Postman

Dans cette partie, nous avons testé les différentes méthodes HTTP de l'API afin de vérifier le bon fonctionnement des opérations CRUD.

4.1 Test de la méthode GET

La méthode GET permet de récupérer la liste de tous les produits enregistrés dans la base de données.

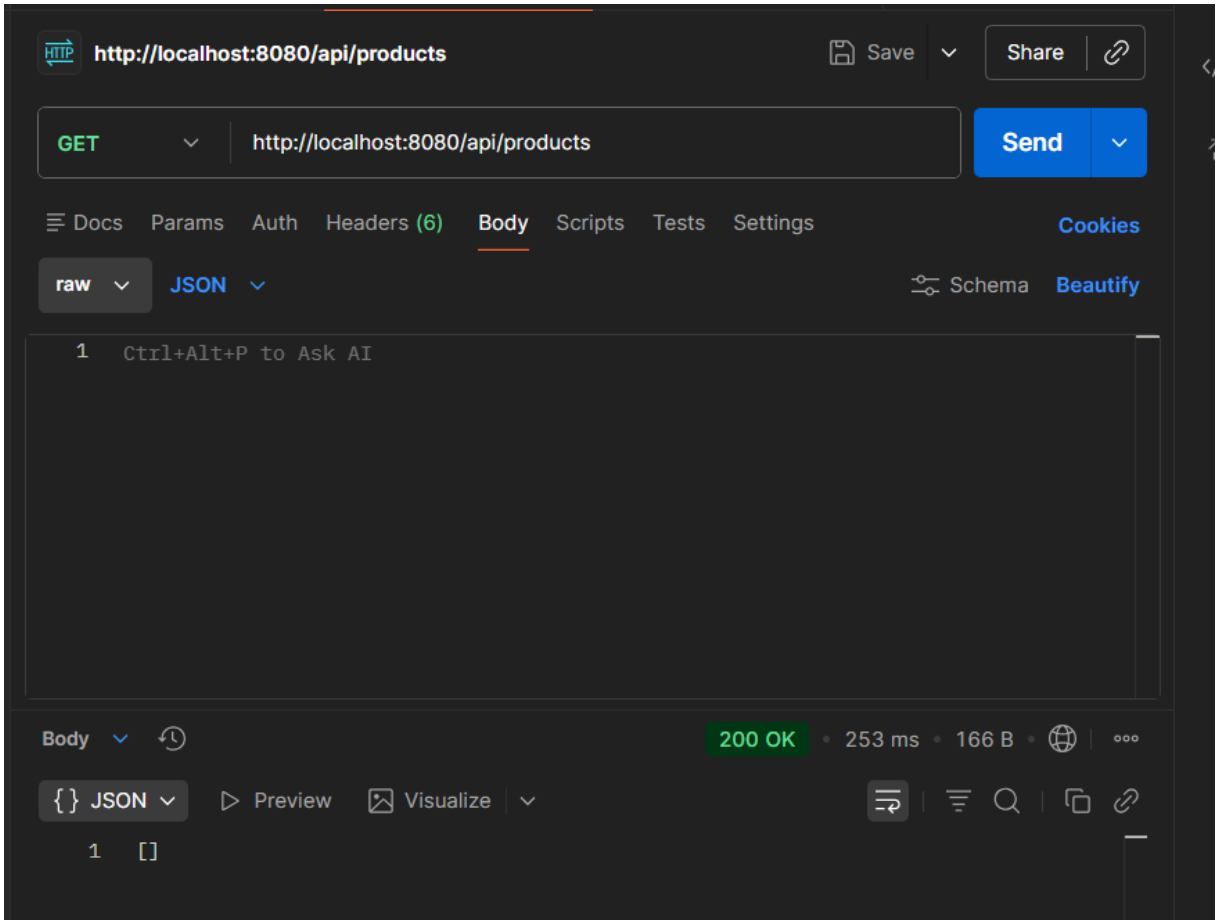

FIGURE 8 – GET /api/products

4.2 Test de la méthode POST

La méthode POST permet d'ajouter un nouveau produit à la base de données en envoyant les informations du produit au format JSON.

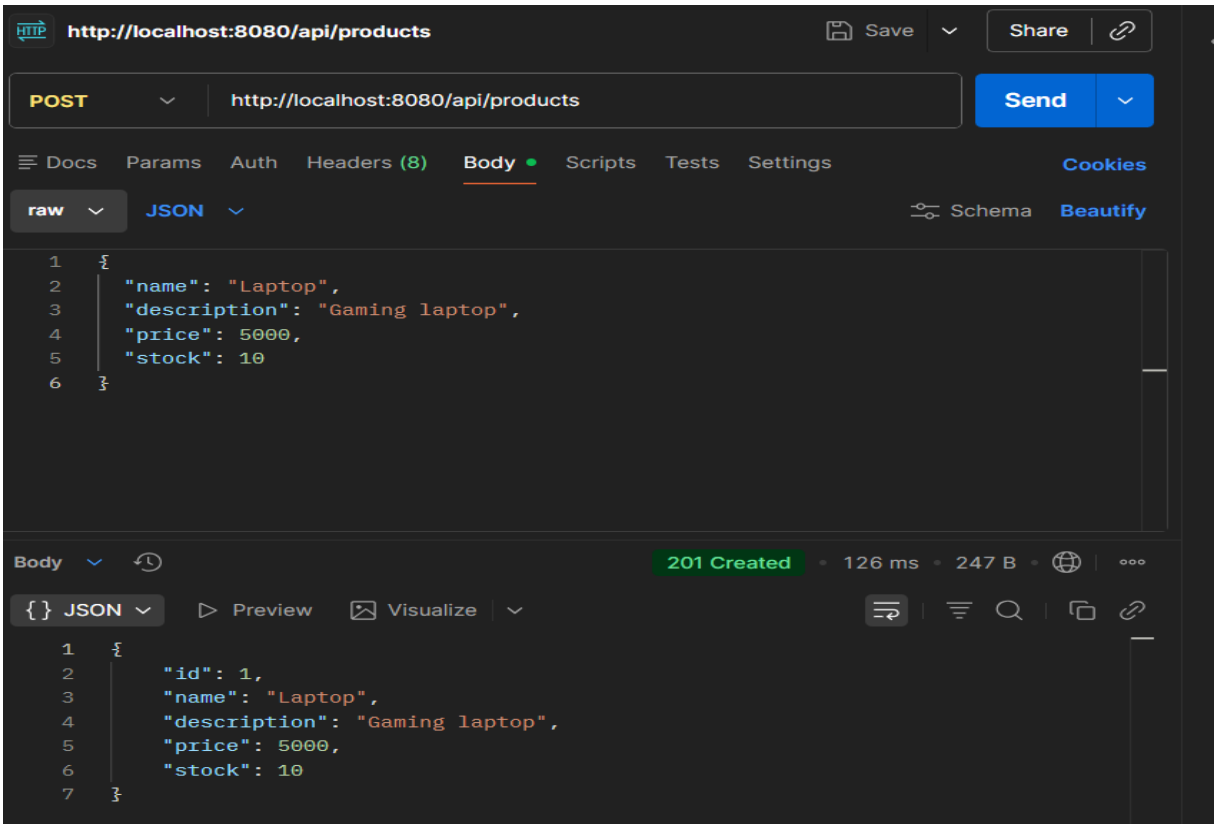

FIGURE 9 – POST /api/products

4.3 Test de la méthode PUT

La méthode PUT permet de modifier un produit existant à partir de son identifiant.

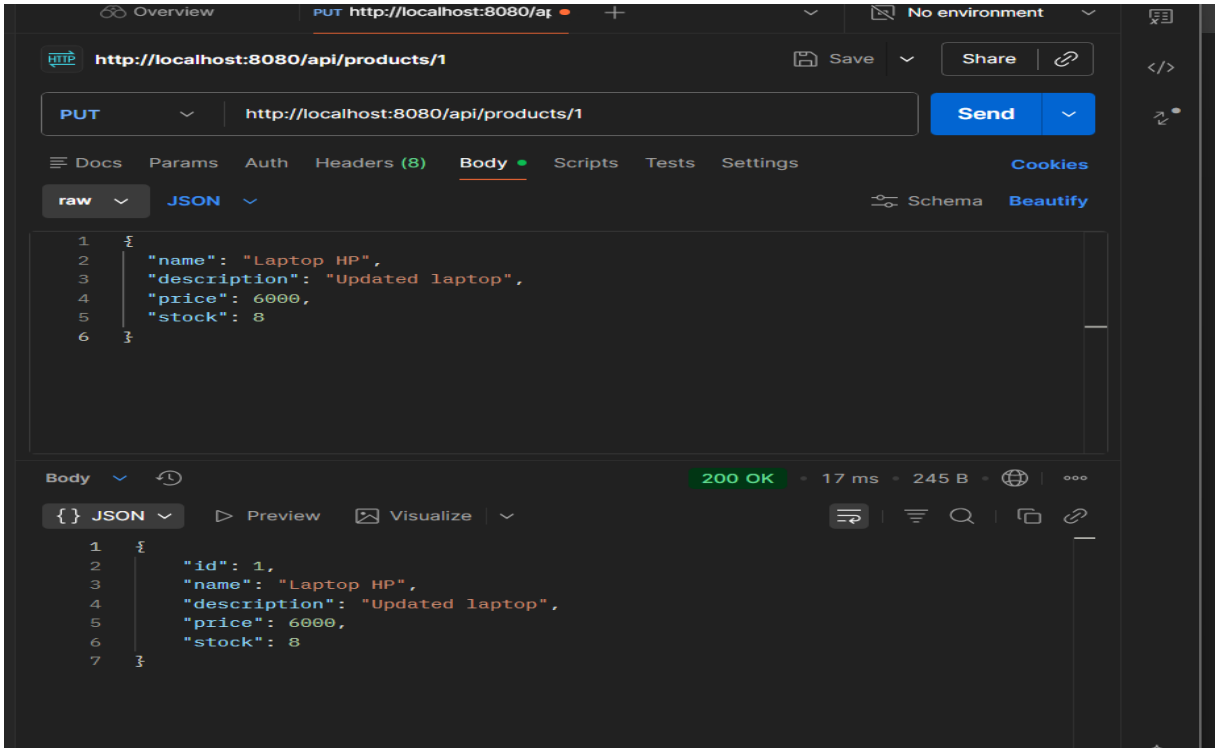

FIGURE 10 – PUT /api/products/{id}

4.4 Test de la méthode DELETE

La méthode DELETE permet de supprimer un produit de la base de données en utilisant son identifiant.

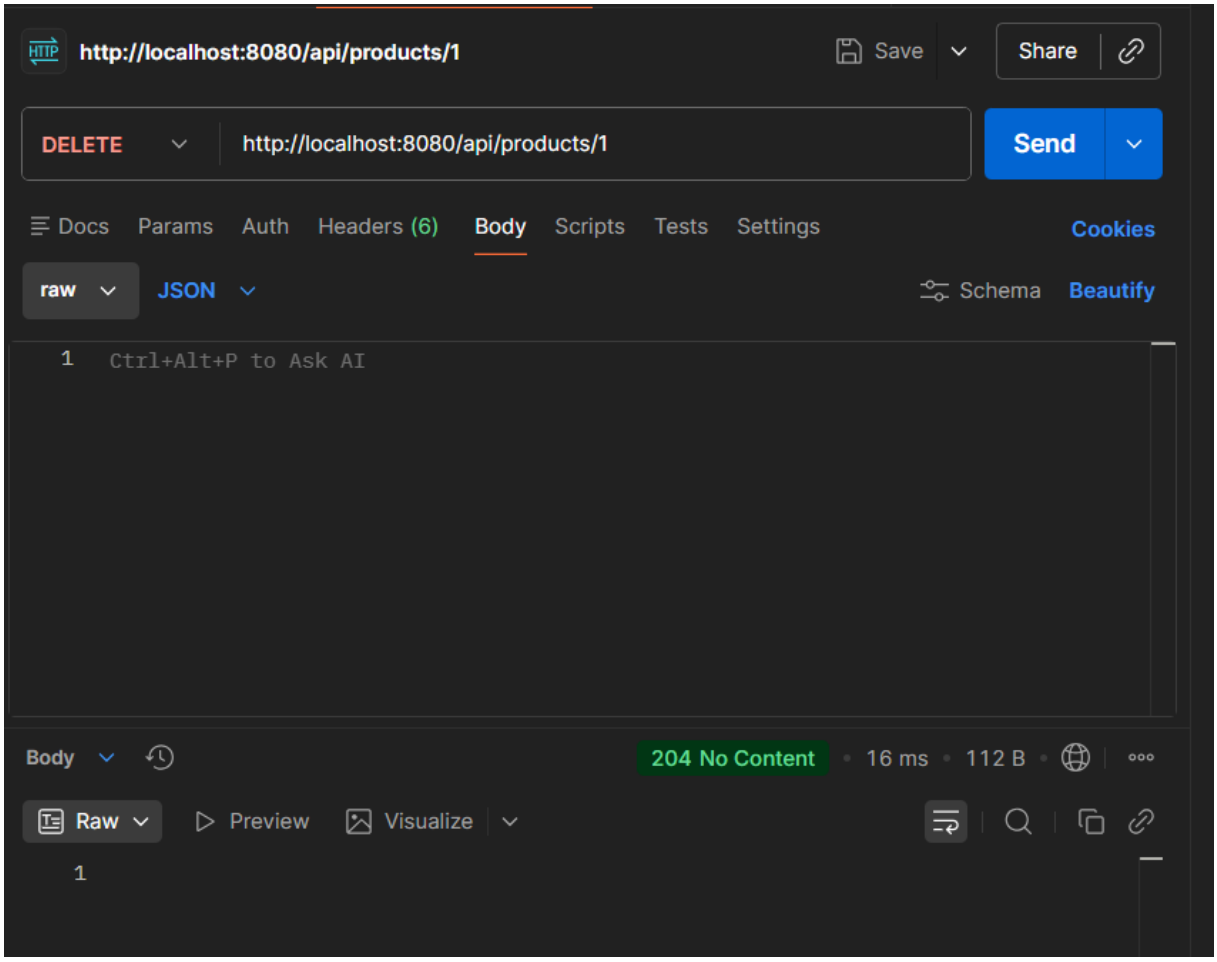

FIGURE 11 – DELETE /api/products/{id}

5 Exercice : gestion des catégories

Dans cette partie, nous avons ajouté une nouvelle entité Category et établi une relation ManyToOne avec l'entité Product. Cette relation permet à plusieurs produits d'appartenir à une même catégorie.

5.1 Création de l'entité Category

Nous avons créé la classe Category contenant les attributs nécessaires à la représentatio d'une catégorie dans la base de données

```java
package com.ecommerce.monolith.product.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Category {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
}
```

FIGURE 12 – Classe Category.java

5.2 Modification de l'entité Product

Nous avons modifié l'entité Product afin d'ajouter la relation ManyToOne avec Category. Ainsi, chaque produit peut être associé à une catégorie via une clé étrangère.

```java
    @ManyToOne
    @JoinColumn(name = "category_id")
    private Category category;
}
```

FIGURE 13 – Ajout de la relation ManyToOne dans Product.java

5.3 Création de CategoryRepository

Nous avons créé CategoryRepository en héritant de JpaRepository. Cette interface permet d'effectuer les opérations de base sur les catégories sans implémentation manuelle.

```java
package com.ecommerce.monolith.product.repository;

import com.ecommerce.monolith.product.model.Category;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CategoryRepository extends JpaRepository<Category, Long> { no usages
}
```

FIGURE 14 – CategoryRepository.java

5.4 Création de CategoryController

Nous avons ensuite créé CategoryController afin d'exposer des endpoints REST pour l'ajout et la récupération des catégories. Cela nous a permis de tester facilement cette entité avec Postman

```java
package com.ecommerce.monolith.product.controller;

import com.ecommerce.monolith.product.model.Category;
import com.ecommerce.monolith.product.repository.CategoryRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/categories")
@RequiredArgsConstructor
public class CategoryController {

    private final CategoryRepository repository;

    @GetMapping
    public List<Category> getAll() {
        return repository.findAll();
    }

    @PostMapping
    public Category create(@RequestBody Category category) {
        return repository.save(category);
    }
}
```

FIGURE 15 – CategoryController.java

5.5 Test avec Postman

Nous avons d'abord créé une catégorie, puis nous avons ajouté un produit lié à cette catégorie. Enfin, nous avons vérifié que la relation entre les deux entités fonctionnait correctement.

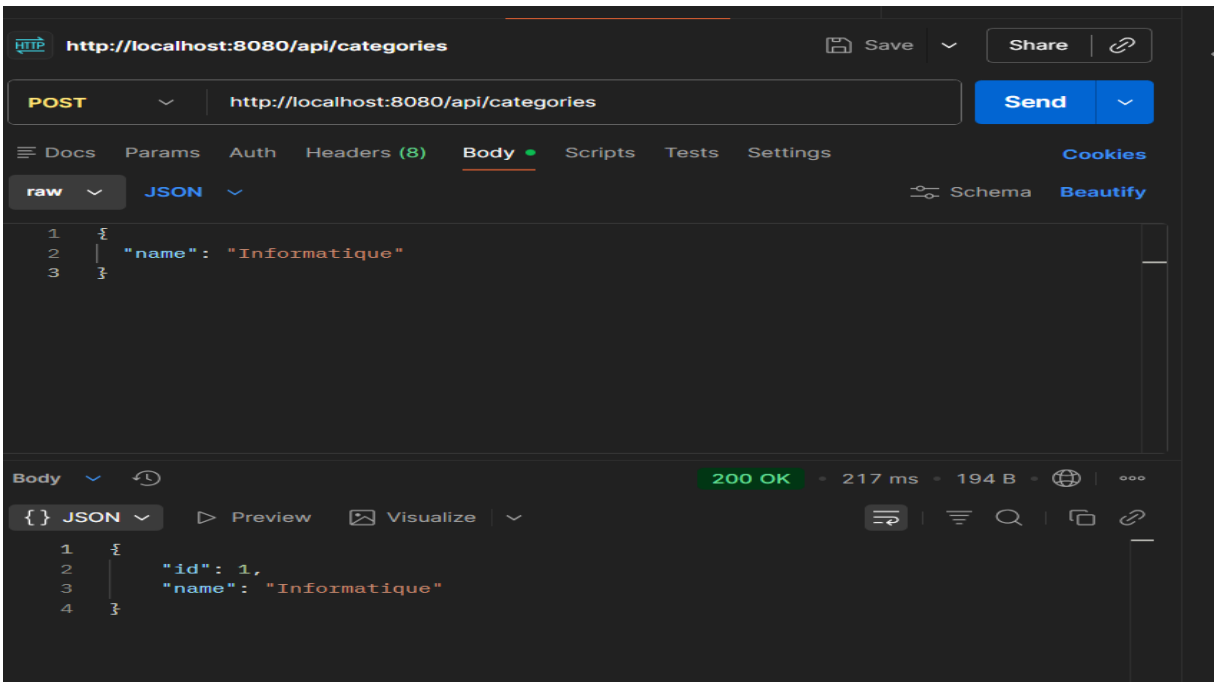

FIGURE 16 – POST /api/categories

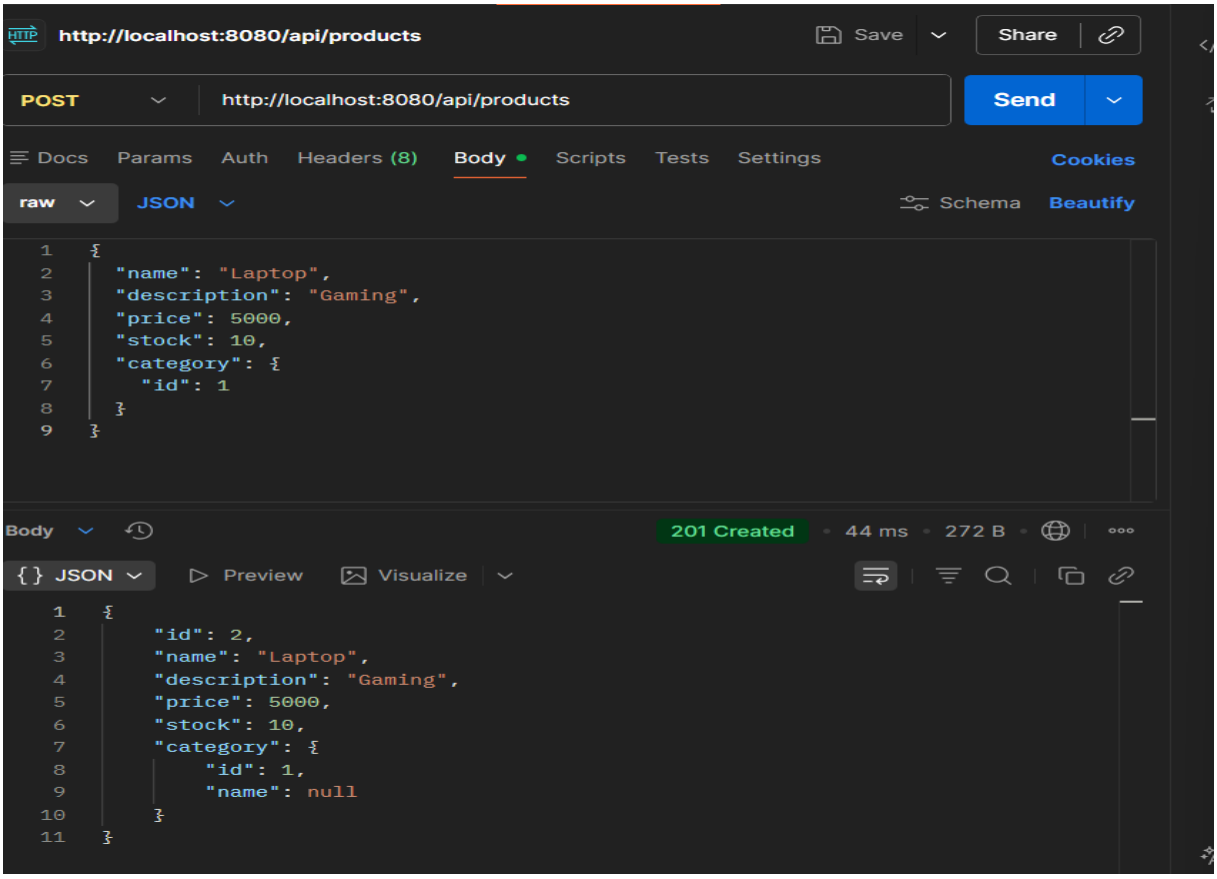

FIGURE 17 – POST /api/products avec une catégorie

1 Conclusion

Ce TP nous a permis de comprendre la création d'une API REST avec Spring Boot, la gestion des opérations CRUD ainsi que la mise en place de relations entre entités avec JPA. L'utilisation de PostgreSQL et de Postman nous a permis de valider le fonctionnement global de l'application, depuis la persistance des données jusqu'au test des endpoints REST.